

Option 1: AI-Powered On-Chain Agent Protocol with Intent Infrastructure

Project Summary

Design and develop a scalable execution network for AI-powered on-chain agents that can accept user intents, batch requests, and execute them efficiently across multiple transactions or workflows. The project should focus on improving throughput and reducing execution bottlenecks for autonomous agents operating on blockchain systems.

Background & Problem Statement

AI agents are emerging as a new type of blockchain user. Unlike normal users, agents may generate a large number of actions, rebalance positions frequently, and interact with multiple protocols automatically. Traditional blockchain execution is not well suited for this pattern because every intent may compete for blockspace, suffer from latency, and face high gas costs.

This project explores how to build a more scalable infrastructure for on-chain agents by introducing intent batching, asynchronous task handling, and parallelizable execution flows. Students should consider how a blockchain system can support large numbers of agent requests without sacrificing correctness or security.

Feature Requirements

- Smart contract or protocol layer for registering and managing autonomous agents.
- Intent submission interface for actions such as swaps, transfers, rebalancing, or repeated scheduled tasks.
- Batching or aggregation mechanism to combine multiple user or agent intents into fewer on-chain executions.
- Execution coordinator or relay design that supports asynchronous processing.
- Performance dashboard or report showing throughput, latency, failed execution rate, and gas cost before and after batching.

Hints & Directions

- Explore ERC-4337, intent-centric architecture, and account abstraction.
- Consider simple agent use cases such as treasury rebalancing, DCA execution, arbitrage monitoring, or auto-compounding.

- Think about how parallel execution or grouped settlement could improve scalability.
- Students may simulate the AI or off-chain intelligence layer if full LLM integration is out of scope.
- Strong projects should discuss trade-offs between decentralization, efficiency, and trust assumptions.

References

1. [ERC-4337 Documentation](#)

A strong starting point for account abstraction, UserOperation, bundlers, paymasters, and smart wallet flow. This is probably the most useful foundation if you want agent-style execution without relying on EOAs.

2. [CoW Protocol – Intents](#)

Good for understanding what an “intent” looks like in practice: users specify desired outcomes, while external solvers determine execution paths.

3. [Anoma Docs](#)

Useful for students who want to study intent-centric architecture at a broader systems level, not just trading. Anoma frames intents as a general-purpose blockchain design primitive.

4. [Across Docs – Crosschain Intents / Intent Architecture](#)

Helpful if students want their agents to work across chains or rollups. Across explains how intent systems can be decomposed into RFQ, relayer, and settlement layers.

5. [dappOS Intent Documentation](#)

Useful for students who want an easy conceptual model of intent-centric execution networks and service-provider-based execution.

Where to Start

1. ERC-4337 smart wallet flow,
2. one concrete intent format,
3. a very small agent use case such as auto-swap, scheduled transfer, or auto-rebalance.

Option 2: Payment Appchain Built on OP Stack

Project Summary

Design and deploy a payment-focused appchain using OP Stack or a similar rollup framework. The appchain should be optimized for high-throughput, low-cost payment transactions and should demonstrate why application-specific chains can outperform general-purpose chains for targeted use cases.

Background & Problem Statement

Payments require fast confirmation, low transaction fees, predictable execution, and simple user experience. General-purpose public blockchains often struggle to meet these needs at scale due to congestion and competition for blockspace from unrelated applications.

Appchains offer a promising solution by specializing blockchain infrastructure for one domain. In this project, students will build a payment appchain and evaluate how rollup-based architecture can improve transaction throughput and cost efficiency for payment use cases.

Feature Requirements

- Deploy a basic payment appchain using OP Stack or a similar framework.
- Implement a payment application supporting transfers, merchant payment records, or recurring payments.
- Define a simple fee model optimized for small-value, high-frequency transactions.
- Build a user interface for sending payments and viewing transaction status.
- Include performance analysis such as TPS, average confirmation time, and fee comparison against Ethereum or another public testnet.

Hints & Directions

- Focus on application-specific optimization rather than building a general-purpose chain.
- You can simulate merchants and repeated transactions to generate payment traffic.
- Consider sequencer design, batching strategy, and withdrawal or settlement assumptions.
- Good projects can discuss whether stablecoin-native payments would be a better design than volatile-token payments.

- Strong teams should explain why OP Stack is appropriate for this use case and where its limitations remain.

References

1. [OP Stack Introduction](#)

The official overview of the OP Stack as a standardized modular framework for building Ethereum Layer 2 rollups. This should be the first read.

2. [OP Stack Docs](#)

Useful for those who actually want to deploy or simulate an OP Stack chain, understand its components, and learn chain operator basics.

3. [Optimism GitHub Repository](#)

Helpful for teams that want to see the actual codebase structure or understand what components exist in a production-grade stack.

4. [ERC-4337 Documentation](#)

Relevant if you want to make the payment appchain more usable with smart wallets, sponsored gas, or better UX for merchants and users.

5. [EigenDA OP Stack Integration Guide](#)

Advanced but useful for strong teams who want to explore whether an OP Stack chain can use an alternative DA layer for better scalability economics.

Where to start

- local or testnet OP Stack setup,
- simple payment contract or stablecoin transfer app,
- benchmark of fee and latency versus Ethereum or Sepolia.

Option 3: Data Availability-Aware Rollup Benchmarking Platform

Project Summary

Build a prototype benchmarking platform that evaluates how different data availability approaches affect the performance and cost of rollup-style blockchain systems. The project should compare several DA assumptions or publication strategies and show how they impact scalability.

Background & Problem Statement

As rollups scale execution, data availability becomes a major bottleneck. Publishing all transaction data to a base chain can be expensive, while alternative DA layers may improve cost efficiency but introduce additional trust or architectural assumptions.

This project allows you to explore one of the most important current topics in blockchain scalability: how to manage data availability efficiently while preserving verifiability and security.

Feature Requirements

- Build a simple rollup simulator or prototype that posts transaction data under different DA assumptions.
- Compare at least two approaches, such as posting all calldata to Ethereum, using compressed data, or using an external DA layer or mock DA layer.
- Measure cost, throughput, and latency under different transaction loads.
- Create a dashboard or visualization showing DA overhead as transaction volume increases.
- Provide analysis of the trade-off between cost savings and trust/security assumptions.

Hints & Directions

- A full real DA integration is not strictly necessary; simulation is acceptable if the design is rigorous.
- Students should focus on the cost of data publication, not just execution.
- This project is especially suitable for teams comfortable with benchmarking and system analysis.

- Good teams can relate findings to Celestia, EigenDA, or Ethereum calldata/blobs.
- Encourage discussion of whether DA is the true scaling bottleneck for some rollup systems.

References

1. [Celestia Docs – Data Availability](#)

A strong conceptual entry point for modular DA, data availability sampling, and why execution can be decoupled from consensus.

2. [EigenDA Overview / Docs](#)

Useful for understanding a modern DA protocol aimed at rollups, with a focus on throughput and lower DA cost.

3. [OP Stack + EigenDA Integration Guide](#)

Helpful for stronger teams that want to see what DA integration looks like in a real rollup stack instead of keeping everything theoretical.

4. [Scroll Rollup Process](#)

Good for understanding where batching, data commitment, proof generation, and finalization fit into an actual rollup pipeline.

5. [“Analyzing and Benchmarking ZK-Rollups” paper](#)

A strong research reference for teams that want a more rigorous benchmarking mindset and clearer terminology around rollup architecture.

Where to start

- Ethereum calldata/blobs model,
- compressed posting,
- mock external DA posting, and then chart cost/latency trade-offs.

Option 4: Modular Appchain with zkEVM Execution Layer or Rollup-Based Execution Layer

Project Summary

Build and evaluate a modular appchain or rollup-style blockchain that separates execution from other core blockchain functions such as consensus and data availability. The project should focus on Ethereum-compatible execution and include a performance study comparing the modular design with a more traditional monolithic deployment.

Background & Problem Statement

A major direction in blockchain scalability is modular architecture. Instead of forcing a single chain to handle execution, consensus, and data availability all at once, modular systems split these responsibilities across different layers. This improves specialization and can significantly increase performance and flexibility.

This project helps you understand how appchains, zkEVMs, and modular rollups improve scalability. It also encourages you to evaluate practical issues such as proving latency, sequencing, data publication, and developer experience.

Feature Requirements

- Deploy a simple modular chain or rollup prototype using a zkEVM stack, OP Stack-style stack, or other modular framework.
- Implement at least one application use case on the chain, such as payments, NFT minting, gaming actions, or simple DeFi interactions.
- Configure or simulate separate components for execution, settlement, and data availability.
- Benchmark key metrics such as transaction throughput, confirmation latency, and gas or fee profile.
- Provide a short report or dashboard explaining the scalability benefits and trade-offs of the chosen architecture.

Hints & Directions

- May use available testnets, local devnets, or framework templates rather than building all infrastructure from scratch.
- Encourage comparison between monolithic execution and modular rollup-based execution.

- Good teams can discuss whether zk rollups or optimistic rollups are more suitable for their chosen use case.
- Explicitly connect your design to the blockchain trilemma.
- Advanced teams may explore how proving delay, DA cost, or centralized sequencing affects overall scalability.

References

1. [OP Stack Documentation](#)

Useful for teams choosing an optimistic-rollup-style modular design.

2. [Scroll Architecture / Rollup Process](#)

Very helpful for students who want to understand a zk-rollup-style architecture in terms of execution, sequencing, proof generation, settlement, and DA.

3. [Polygon zkEVM Docs](#)

A good entry point for students exploring zkEVM architecture and Ethereum equivalence, though note Polygon zkEVM is being sunset during 2026, so it is better used as a learning reference than as a fresh platform bet.

4. [Celestia Data Availability Docs](#)

Useful if students want to model modularity more explicitly by separating execution from DA.

5. [EigenDA Overview](#)

Another relevant source for teams studying modular DA as part of their appchain or rollup design.

Where to start

- choose **one** stack,
- deploy or simulate **one** app,
- compare modular vs monolithic on a small number of metrics, instead of trying to build a whole production chain.

Option 5: Parallel Transaction Execution Engine for EVM

Workloads

Project Summary

Design and implement a prototype system that identifies independent transactions or smart contract calls and executes them in parallel, or simulates parallel execution, to improve throughput for EVM-based workloads.

Background & Problem Statement

Most blockchains execute transactions sequentially, even when many transactions do not actually conflict with one another. This creates a major scalability bottleneck. Parallel execution is a promising direction for improving performance, but it introduces challenges in conflict detection, deterministic replay, and state consistency.

This project helps you study one of the core frontiers in blockchain execution scaling by building a small-scale prototype or experimental framework.

Feature Requirements

- Implement or simulate a transaction scheduler that identifies non-conflicting transactions.
- Support parallel or pseudo-parallel execution for selected workloads.
- Define a conflict detection rule based on account access, storage slot access, or simplified read/write sets.
- Compare throughput and execution time against sequential processing.
- Produce a report explaining when parallel execution helps and when it does not.

Hints & Directions

- Students do not need to modify Ethereum clients from scratch; a simulator or simplified execution engine is acceptable.
- Good workloads may include token transfers, NFT minting, payment operations, or independent contract calls.
- Students should pay attention to determinism and rollback when conflicts occur.

- Strong projects can discuss links to Aptos-style parallelism, SVM-style execution, or parallel EVM efforts.
- This topic is ideal for students interested in system design rather than only frontend dApp development.

References

1. [Block-STM Paper](#)

Probably the most important reading for this option. It explains speculative parallel execution, conflict handling, and throughput gains in blockchain systems.

2. [Aptos Developer Documentation](#)

Useful background for students who want to understand how a modern high-throughput chain positions its execution model.

3. [Solana Sealevel Article](#)

A classic reference on account-based parallelism and why explicit read/write sets matter for runtime parallelization.

4. [Solana Docs](#)

Helpful for students who want broader context on a production system built around high-throughput execution.

5. [ERC-4337 Documentation](#)

Not a parallel-execution source by itself, but useful if teams want to think about how higher-level transaction packaging or user operations could feed into a scheduler.

Where to start

- build a simulator
- define a conflict rule
- run sequential vs parallel batches
- explain when rollback happens

Option 6: Cross-Rollup Intent Router for Scalable User Transactions

Project Summary

Build a cross-rollup routing system that allows users to submit a high-level transaction intent and automatically routes the request to the most suitable rollup or execution environment based on fee, latency, or availability.

Background & Problem Statement

As the ecosystem becomes increasingly modular, users will face a fragmented environment with many rollups, appchains, and settlement paths. Manually deciding where to transact introduces friction and limits usability. A scalable blockchain future may require intent-based routing systems that abstract away chain choice from end users.

This project focuses on the infrastructure needed to make a multi-rollup ecosystem usable and scalable.

Feature Requirements

- Design an intent interface for users to specify desired outcomes such as payment, token swap, or asset transfer.
- Implement a routing layer that selects between at least two execution environments, whether real or simulated.
- Include routing criteria such as cost, latency, congestion, or execution success probability.
- Build a frontend showing route selection and execution status.
- Analyze how this architecture improves user scalability and system efficiency.

Hints & Directions

- Students may simulate different rollups if direct integration with multiple live systems is too difficult.
- This project can combine ideas from account abstraction, intents, and modular blockchains.
- Good teams will explain where trust assumptions lie in the router or relayer.
- Strong projects may discuss whether intent routing creates new centralization risks.
- The key is not only routing logic, but also the scalability argument behind it.

References

1. [Across Docs – Crosschain Intents / Intent Architecture](#)

One of the best practical references for this topic because it directly explains intent-based cross-chain execution and routing layers.

2. [CoW Protocol – Intents](#)

Useful for understanding how users express desired outcomes and how external solvers handle execution.

3. [Anoma Docs](#)

Helpful for students who want a broader architectural view of intents and interoperability in a fragmented ecosystem.

4. [NEAR Intents Overview](#)

A nice additional reference showing that intents are becoming a broader multichain UX pattern, not just a DEX-specific idea.

5. [OP Stack Docs](#)

Useful if students want their router to target multiple OP Stack-style chains or use rollup characteristics such as cost and latency in route selection.

Where to start

- user submits “I want X outcome,”
- router compares 2 simulated rollups
- chooses based on fee + latency,
- frontend shows why the route was chosen.

Option X: BYOP: Propose Your Own Project

Note: You MUST get an approval from the course coordinator about the project ideas on their difficulties, scopes, and other factors before proceeding with the development project.

- You will be required to submit a one-page write up on your project idea, similar to the other topics described in this document.
- The write-up should contain the background and problem statement, plus what your MVP will be in terms of feature requirements.
- The write-up may also contain your choice of bonuses/nice to have features.

Here are some ways to formulate your idea if you don't know where to start:

1. Read the [DeFi SoK paper](#) (first 3 chapters) and go to [top DeFi projects](#)' GitHub repo to find a feature request in their backlog that involves a reasonable amount of contract development.
 - o Good example: Order-book based DEX, AMM based DEX type of tasks/features.
 - o Bad example: Simple "increase testing coverage" or "fix a certain bug" tasks.
2. Watch Vitalik's "[Things that matter outside DeFi](#)" talk for more ideas like the following ones.
 - o Censorship resistant, high-quality engagement social media – mostly mechanism design (read [quadratic voting](#))
 - o NFTs
 - o Authenticate/attestation with Eth identity – self-sovereign & social recovery (Log in with Eth, see [EAuth EIP](#))
 - o Retroactive public good funding, (optimism's [blog post](#))
3. Check [Bitcoin bounties](#) or hackathon to pick a topic with reasonable contract development.

Notes: Please make sure the BYOP aligns with the course,

Ensure that your project idea directly relates to the core themes and skills taught in the course, such as:

- Blockchain Scalability: Can your project propose a novel solution to blockchain scalability issues, or significantly improve upon existing solutions?

- Blockchain Trilemma: Does your idea offer a new perspective on balancing the trilemma of security, scalability, and decentralization?
- Smart Contracts and Decentralization: How does your project leverage smart contracts or decentralized systems to solve the identified problem?
- Privacy-Scalability Trade-offs: If your project involves handling data, how does it navigate the trade-offs between user privacy and system scalability?

General Tips for All Projects:

- Start with a design phase where you outline the project's architecture and main components.
- Break down the project into smaller, manageable tasks and assign them within the group.
- Use agile development practices like sprints to keep the project on track.
- Regularly test your work to catch and resolve issues early.

Given the ambitious nature of these projects and the tight timeline, teams should focus on delivering a minimum viable product (MVP) that demonstrates the core functionality. Further enhancements can be proposed as future work in the project's documentation.